Ask or Act for Cooperative Assistance via Inverse Planning

CS6208 Final Report

Zhu Yufan A0238993J *April 10, 2026*

Abstract

We study cooperative assistance under instruction ambiguity: should a helper ask a clarifying question or act immediately? We implement a controlled gridworld in which an assistant infers a Bayesian posterior over a principal's hidden goal from an ambiguous instruction and observed actions, then decides whether to act or ask. Our *AskOrAct* policy selects whichever option minimises expected team cost, using a value-of-information criterion. Across 19,800 evaluation episodes spanning ambiguity levels $K \in \{1, 2, 3, 4\}$, rationality parameters, and principal noise, *AskOrAct* achieves $+7.9$ pp and $+8.4$ pp higher success than a non-asking baseline at $K{=}3$ and $K{=}4$ (95% CI both exclude zero), while asking only 0.6–0.7 questions per episode. Robustness sweeps confirm the advantage persists under answer noise and principal model mismatch. Ablations reveal that a non-zero question cost and correct ask-window semantics are necessary for stable performance.

1 Introduction

Cooperative assistance requires a helper to decide between acting on its current belief or seeking more information. Acting immediately risks committing to the wrong goal; asking too often wastes time and may frustrate the principal. The optimal policy depends on the informativeness of questions, the cost of a wrong action, and the principal's revealed behaviour.

We study this problem in a controlled gridworld where the principal's goal is hidden and the instruction is ambiguous (K candidate goals map to the same surface form). The assistant maintains a Bayesian posterior over goals, updated from the instruction prior and observed principal actions via inverse planning. At each step, the *AskOrAct* policy computes the expected cost of acting now (*CostAct*) and the expected cost of asking the best available question (*CostAsk*), and selects the lower-cost option.

The main contributions are:

- A reproducible cooperative gridworld benchmark with templated ambiguity and a Boltzmann principal model.
- A value-of-information ask-or-act policy with entropy gating, ask-window, and per-question cost.
- A systematic evaluation across 5 core policies, plus targeted stronger baselines at $K{=}2$, ablations, and robustness sweeps with 95% bootstrap CIs.
- An analysis of when asking helps and when it fails.

2 Related Work

Inverse planning treats observed behaviour as evidence for latent goals via a rational action model [1]. We build on this perspective for online goal inference during cooperation.

Cooperative Inverse Reinforcement Learning [2] formalises human-robot teams where the robot is uncertain about the human's reward function. Our setting is complementary: we use a fixed, known reward structure but allow the principal's goal to be uncertain from a known template family.

CLIPS [5] motivates combining language and action evidence for goal inference in richer environments. We study the same combination in a controlled setting where the relative contribution of instruction vs. action evidence can be measured.

Active information gathering in POMDPs [4] motivates asking as a sensor action. Our approach approximates the POMDP solution with a one-step lookahead cost comparison, which is tractable and interpretable.

Decision-theoretic models of question asking [3] connect utility and effort to communication. Our question cost term directly operationalises this: asking is beneficial only when the expected posterior improvement outweighs the cost.

3 Method

3.1 Environment

We use a $9{\times}9$ two-room gridworld with 6 object instances. A principal has a hidden goal object. An instruction template maps the goal to a surface form shared by K candidate goals (ambiguity level K). The principal takes approximately rational actions towards its goal, sampled from a Boltzmann policy with rationality β and noise ε:

$$\pi_{\beta,\varepsilon}(a \mid s, g) \propto \exp(\beta \cdot q(s, a, g)) \cdot (1{-}\varepsilon) + \varepsilon/|A|.$$

The assistant observes the state, instruction, and all principal actions, but not the true goal. An episode ends when the assistant picks an object (success if correct) or the step budget is exhausted. Figure 1 shows the full cooperative setup, including the ambiguous instruction, the assistant's partial information, and the question menu used for clarification.

3.2 Bayesian Posterior

The assistant maintains a posterior $b_t(g)$ over candidate goals $\mathcal{G}$. Initialisation uses the instruction likelihood under each goal. After observing principal action a_t at state s_t:

$$b_{t+1}(g) \propto b_t(g) \cdot \pi_{\hat{\beta},\hat{\varepsilon}}(a_t \mid s_t, g),$$

where $\hat{\beta}, \hat{\varepsilon}$ are the assistant's assumed principal parameters.

When a question q is asked and answered α, the posterior is updated by the answer likelihood:

$$b(g) \propto b(g) \cdot P(\alpha \mid q, g, \text{noise}),$$

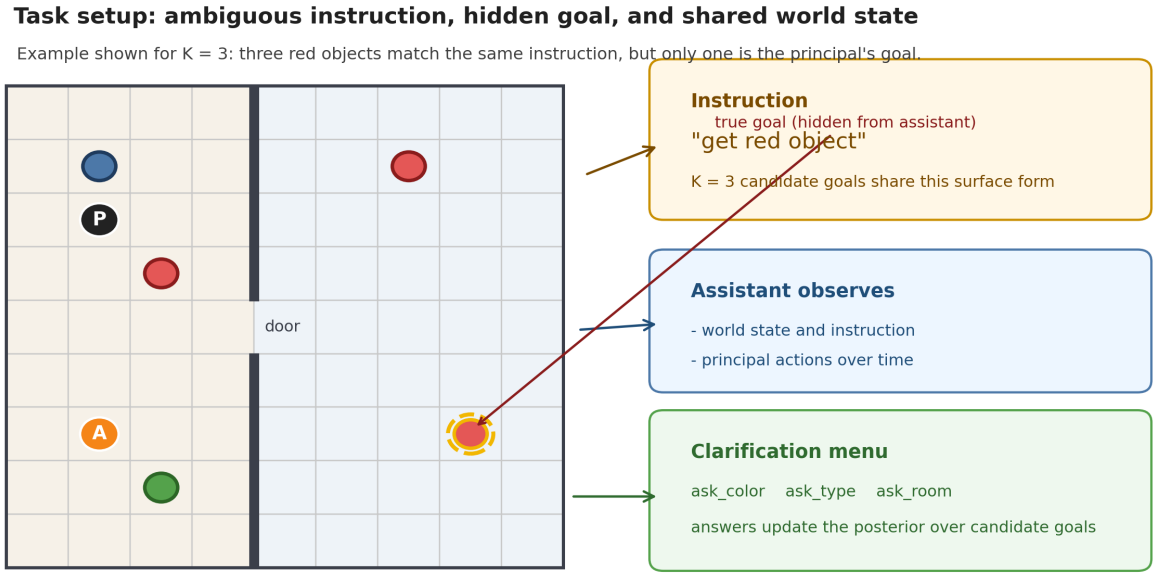

Figure 1: Task setup. The principal has a hidden goal object in a two-room gridworld, while the assistant sees the shared world state, an ambiguous instruction, and the principal's movement. Clarification questions provide structured observations that can collapse the posterior over candidate goals.

where answer noise is per-type: color 0.05, room 0.10, object 0.20.

3.3 Ask-or-Act Policy

At step t, the assistant computes:

$$\text{CostAct} = \sum_g b_t(g) \cdot \text{dist}(s_t, g),$$

the expected path length to a goal sampled from the current posterior, and

$$\text{CostAsk}(q) = 1 + c_q + \mathbb{E}_\alpha[\text{CostAct after update from } \alpha],$$

where $c_q = 0.5$ is the question cost. The policy asks $\arg\min_q \text{CostAsk}(q)$ if $\min_q \text{CostAsk}(q) < \text{CostAct}$, subject to:

1. **Entropy gate**: skip ask branch if $H(b_t) \leq 0.3$ (posterior already concentrated).
2. **Ask window**: do not ask after step 6 (too late to benefit).
3. **Budget**: at most $Q_{\max} = 3$ questions per episode.

Figure 2 summarises this decision process from instruction grounding through posterior update and the final ask-versus-act comparison.

3.4 Baselines

NeverAsk: always acts toward the MAP goal. **AlwaysAsk**: asks the highest-information-gain question until budget is exhausted, then acts. **InfoGainAsk**: like AlwaysAsk but gated by entropy threshold and information-gain threshold. **EasyInfoGainAsk**: InfoGainAsk limited to one question per episode. **RandomAsk**: same gating as InfoGainAsk but picks a random question. **POMCPPlanner**: POMDP baseline using PO-UCT / POMCP tree search over ask and act actions. In the main sweep, **EasyInfoGainAsk** and **POMCPPlanner** are evaluated only at K=2 as targeted stronger baselines; the 5 core policies above are evaluated across all (K, ε, β) cells.

4 Experimental Setup

Sweep. Main evaluation: $K \in \{1, 2, 3, 4\}$, $\varepsilon \in \{0, 0.05, 0.1\}$, $\beta \in \{1, 2, 4\}$, 5 replicate seeds $\times$ 20 episodes per condition = 100 episodes per cell. The 5 core policies are evaluated in every cell; *EasyInfoGainAsk* and *POMCPPlanner* are additionally evaluated at K=2. Total: 19,800 episodes.

Metrics. *Success rate*: fraction of episodes where the assistant picks the true goal. *Regret*: team cost − oracle cost (oracle knows the true goal). *Questions asked*: average per episode. *MAP correct*: fraction where the MAP goal matches the true goal.

Uncertainty. 95% bootstrap confidence intervals over episodes (B=1000 resamples).

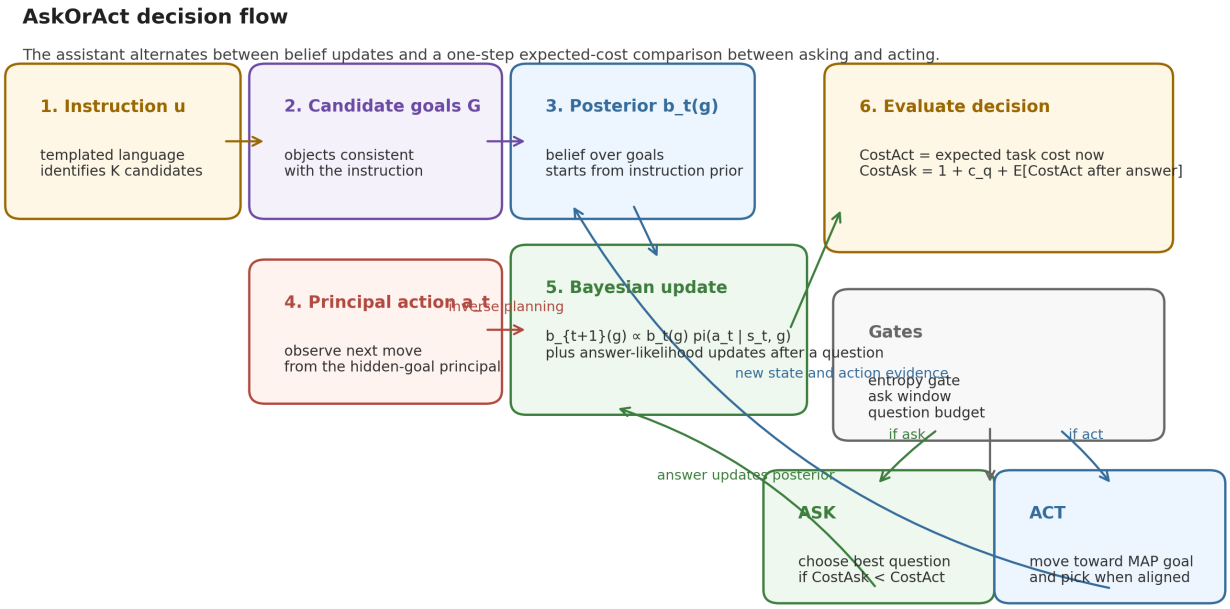

Figure 2: AskOrAct decision flow. The assistant combines instruction-derived candidates with inverse-planning evidence from the principal's actions, updates its posterior, and compares immediate acting against asking the best available clarification question under a shared team-cost objective.

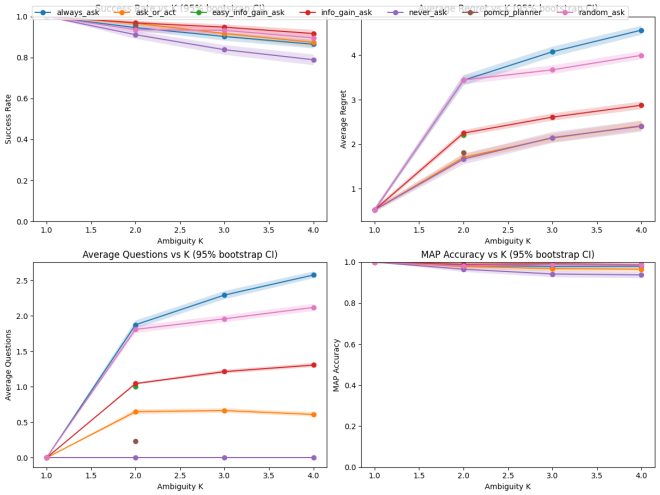

Figure 3: Success rate, regret, and questions asked by ambiguity K (averaged over ε, β). Error bars show 95% CI.

Table 1: Mean results averaged over all (ε, β) conditions. **Bold**: best success per K.

K	Policy	Success	Regret	Q/ep
2	AskOrAct	96.7%	1.71	0.65
	InfoGainAsk	**96.9%**	2.25	1.05
	AlwaysAsk	94.6%	3.44	1.87
	NeverAsk	91.1%	1.67	0.00
3	**InfoGainAsk**	**94.8%**	2.61	1.21
	AskOrAct	91.7%	**2.14**	0.67
	AlwaysAsk	90.2%	4.08	2.29
	NeverAsk	83.8%	2.15	0.00
4	**InfoGainAsk**	**91.7%**	2.88	1.31
	AskOrAct	87.3%	**2.42**	0.61
	AlwaysAsk	86.4%	4.56	2.58
	NeverAsk	78.9%	2.40	0.00

5 Results

5.1 Main Sweep

Figure 3 shows results averaged across all ε and β conditions. At $K{=}1$ (no ambiguity) all policies perform identically: 100% success, zero questions. As ambiguity increases, policies diverge.

Table 1 summarises the key aggregate results.

AskOrAct vs. NeverAsk. At $K{=}3$, *AskOrAct* achieves $+7.9$ pp success (95% CI $[5.0, 10.8]$). At $K{=}4$, $+8.4$ pp (95% CI $[5.1, 12.1]$). Both CIs exclude zero, confirming statistical significance. Crucially, *AskOrAct* asks only 0.61–0.67 questions per episode — it asks selectively.

AskOrAct vs. AlwaysAsk. *AlwaysAsk* asks 2.3–2.6× more questions but achieves lower or similar success at $K{=}3, 4$, and pays much higher regret ($+1.94$ at $K{=}4$). This confirms

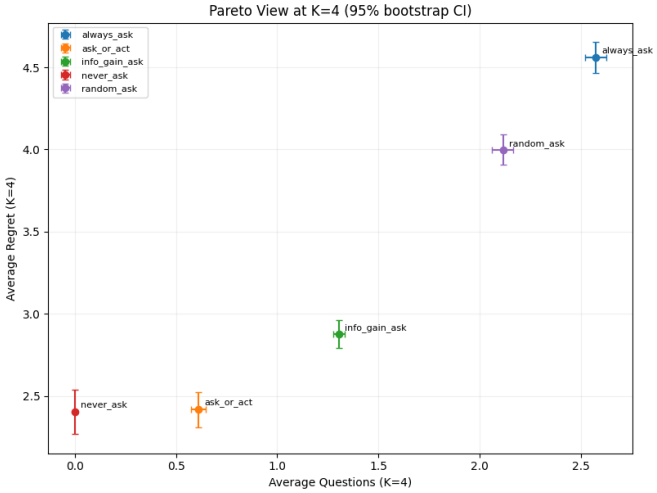

Figure 4: Success rate vs. regret Pareto frontier at $K{=}4$. *AskOrAct* lies on the efficient frontier.

over-asking is costly.

AskOrAct vs. InfoGainAsk. *InfoGainAsk* achieves higher success ($+3.1$–$+4.4$ pp at $K{=}3, 4$) but asks $\sim 2\times$ more questions. *AskOrAct* achieves lower regret at $K{=}3$ (2.14 vs. 2.61), trading peak success for fewer questions.

5.2 Pareto Frontier

Figure 4 shows the Pareto frontier at $K{=}4$. *AskOrAct* is Pareto-efficient: it dominates *NeverAsk* in success while matching its regret, and dominates *AlwaysAsk* and *RandomAsk* in regret.

5.3 Robustness

Answer noise. We sweep per-question answer noise uniformly from 0 to 0.2 (disabling per-type defaults so the swept value applies globally). At $K{=}4$, the *AskOrAct* success advantage over *NeverAsk* holds across all noise levels ($+4$–$+9$ pp). At $K{=}3$, the advantage is smaller and briefly reverses at noise $= 0.1$ (*AskOrAct* 92.0% vs. *NeverAsk* 94.0%); the gap re-opens at 0.2 (95.0% vs. 94.0%), suggesting a moderate noise regime where noisy answers can mislead the posterior.

Principal model mismatch. The assistant's model is fixed at $\hat{\beta}{=}2$ while the principal's true β is varied over $\{1, 2, 4\}$. The hardest condition is $\beta_{\text{true}}{=}1$ (near-random principal), where *AskOrAct* retains $+13.5$ pp over *NeverAsk* (83.5% vs. 70.0%). The largest parameter distance ($\beta_{\text{true}}{=}4$, distance$=2$) is less disruptive because a highly rational principal is easier to model.

Held-out instruction templates. Evaluating on templates not seen during policy design shows mild degradation but preserves policy ordering, confirming results are not template-specific.

Scale-K. Extending to $K \in \{5, 6\}$ shows the asking advantage grows: *AskOrAct* retains meaningful success while *NeverAsk* falls substantially, consistent with VoI being most valuable under high ambiguity.

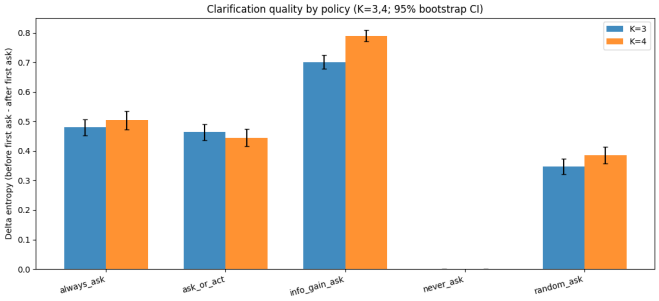

Figure 5: Posterior entropy before and after the first question, by policy and K. *AskOrAct* reduces entropy selectively.

5.4 Ablations

Table 2: Ablation results for *AskOrAct* across mode variants (averaged over K, ε, wrong-pick, deadline conditions).

Mode	c_q	Ask-step	Success	Q/ep
A	0.0	yes	69.2%	1.61
B	0.5	no	73.6%	0.64
C	0.5	yes	73.8%	0.65

Table 2 shows three ablation variants.

Mode A ($c_q{=}0$, no question cost): the policy asks far more (1.61 q/ep) but achieves lower success (69.2%). With zero cost, *CostAsk* is almost always preferred, leading to excessive asking that uses up the ask budget on low-value questions and leaves the posterior insufficiently converged before acting.

Modes B and C differ in whether asking counts as an environment step. Performance is nearly identical (73.6% vs. 73.8%), suggesting the ask-counts-as-step setting has minimal effect once question cost is non-zero.

The ablation confirms that a non-zero question cost is necessary to prevent over-asking and maintain a useful ask-or-act tradeoff.

5.5 Clarification Quality

Figure 5 shows that *AskOrAct*'s first question reduces posterior entropy substantially at $K{=}3, 4$, achieving comparable entropy reduction to *InfoGainAsk* while asking less frequently. At $K{=}1, 2$, *AskOrAct* rarely asks (entropy already low), confirming the entropy gate works as intended.

6 Discussion

When does asking help? The results show asking is most valuable at $K{=}3, 4$, where the uniform prior over goals has high entropy and a single question can resolve 1–2 bits of uncertainty. At $K{=}1$, no policy needs to ask. At $K{=}2$, the gain from asking is moderate ($+5.6$ pp) and asking is expensive relative to the ambiguity.

Limitations. The question bank is fixed and small (3 question types). The answer model is noisy but parameterised; in real settings, answers may be open-ended. The gridworld is small enough for exact posterior enumeration; scaling to larger state spaces would require approximate inference. The principal is simulated; real principals may not follow a Boltzmann model.

Connection to course themes. *AskOrAct* operationalises theory of mind (maintaining beliefs about another agent's goal), inverse planning (updating beliefs from observed behaviour), and cooperative assistance (jointly minimising team cost). Asking is modelled as a cooperative communication act with a cost and an expected benefit [3].

7 Conclusion

We presented *AskOrAct*, a value-of-information policy for cooperative assistance under instruction ambiguity. In a controlled gridworld evaluation, it achieves statistically significant success improvements over a non-asking baseline ($+7.9$ pp at $K{=}3$, $+8.4$ pp at $K{=}4$) while asking fewer than one question per episode on average. Ablations show that a non-zero question cost is critical, and robustness sweeps confirm the advantage persists under answer noise and principal model mismatch. The approach is reproducible and fully deterministic given the provided seeds.

AI Tool Declaration

I used Claude Sonnet 4.6 (Anthropic) for coding assistance, debugging, and LaTeX editing throughout the project. I am responsible for all research decisions, experimental design, and the content of this report.

References

[1] Chris L. Baker, Rebecca Saxe, and Joshua B. Tenenbaum. Action understanding as inverse planning. *Cognition*, 113 (3):329–349, 2009.

[2] Dylan Hadfield-Menell, Anca D. Dragan, Pieter Abbeel, and Stuart Russell. Cooperative inverse reinforcement learning. In *Advances in Neural Information Processing Systems*, 2016.

[3] Julian Jara-Ettinger, Hyowon Gweon, Laura E. Schulz, and Joshua B. Tenenbaum. The naive utility calculus. *Trends in Cognitive Sciences*, 20(8):589–604, 2016.

[4] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[5] Zhi-Xuan Tan, Lexing Ying, Vikash Mansinghka, and Joshua B. Tenenbaum. Pragmatic instruction following and goal assistance via cooperative language guided inverse planning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2024.